

Les fonctions



Définition d'une fonction

- Une fonction est définie par son nom et le mot clé function

```
<?php
function foo ($arg_1, $arg_2, ..., $arg_n) {
    echo "Exemple de fonction.\n";
    return $retval;
}
?>
```

- Des informations peuvent être passées à une fonction en utilisant un tableau d'arguments, dont chaque élément est séparé par une virgule. Un élément peut être une variable ou une constante.
- On appelle paramètres formels les paramètres définis dans l'en-tête d'une fonction et paramètres effectifs les paramètres fournis lors de l'appel d'une fonction



Encapsulation



- Une fonction peut contenir une autre fonction

```
<?php
function foo()
{
    function bar()
    {
        echo "Je n'existe pas tant que foo() n'est pas appelé.\n";
    }
}
/* Impossible d'appeler bar() ici car il n'existe pas. */
foo();
/* Maintenant, nous pouvons appeler bar(),
car l'utilisation de foo() l'a rendu accessible. */
bar();
?>
```



Passage d'arguments (1/3)



- Par défaut, les arguments sont passés à la fonction par valeur (donc vous pouvez changer la valeur d'un argument dans la fonction, cela ne change pas sa valeur à l'extérieur de la fonction).
- Si vous voulez que vos fonctions puissent changer la valeur des arguments, vous devez passer ces arguments par référence. Dans l'exemple ci-dessous, le passage est effectué par l'utilisateur. C'est un passage par référence ponctuel.

```
<?php
function foo ($bar) {
    $bar .= ' , et un peu plus.';
}
$str = 'Ceci est une chaîne';
foo ($str);
echo $str; // affiche 'Ceci est une chaîne'
foo (&$str);
echo $str; // affiche 'Ceci est une chaîne, et un peu plus.'
?>
```



Passage d'arguments (2/3)



- Si vous voulez qu'un argument soit toujours passé par référence, vous pouvez ajouter un '&' devant l'argument dans la déclaration de la fonction :

```
<?php
function add_some_extra(&$string) {
    $string .= ', et un peu plus.';
}
$str = 'Ceci est une chaîne';
add_some_extra($str);
echo $str; // affiche 'Ceci est une chaîne, et un peu plus.'
?>
```



Passage d'arguments (3/3)



- Vous pouvez définir comme en C++ des valeurs par défaut pour les arguments de type scalaire

```
<?php
function servir_apero ($type = "ricard") {
    return "Servir un verre de $type.\n";
}
echo servir_apero();
echo servir_apero("whisky");
?>
```

- Il est à noter que si vous utilisez des arguments avec valeur par défaut avec d'autres sans valeur par défaut, les premiers doivent être placés à la suite de tous les paramètres sans valeur par défaut. Sinon, cela ne fonctionnera pas.

php Portée des Variables



- La portée d'une variable dépend du contexte de sa définition
- Variables globales : définies à l'extérieur d'une fonction. Pour y accéder dans une fonction utiliser `global` ou `$GLOBALS` tableau associatif (avec comme clé le nom de la variable globale)
- Variables locales : définies à l'intérieur de la fonction et les paramètres
- Variables statiques : variables locales conservant leur valeur entre les appels de la fonction

```
function compte () {  
    static $compteur = 0 ;  
    $compteur++ ;  
    echo "Scompteur " ;  
    if ($compteur < 10) compte() ;  
}  
compte() ;
```

Affiche 1 2 3 4 5 6 7 8 9 10.

```
<?php  
$nom="Toto"; /* portée globale */  
include "base.php"; /* dans ce fichier $nom est toujours visible */  
function test()  
{  
    echo $nom; /* portée locale */  
}  
test(); /* n'affiche rien puisque $nom n'a pas de valeur */  
?>
```

```
function test()  
{  
    echo $GLOBALS[nom]; /* globale */  
}
```

php Fonctions pour les variables



- **gettype (\$nom_var)** : détermine le type de données de la variable ("integer", "double", "string", "array", "object", "class" ou "unknown type")
- **settype (\$nom_var, "type")** : définit explicitement le type de variable
- **isset (\$nom_var)** : sert à savoir si une variable possède une valeur (true ou false)
- **unset (\$nom_var)** : détruit une variable
- **empty (\$nom_var)** : renvoie true si la variable est vide ou vaut 0, sinon renvoie false
- On peut également tester un type particulier à l'aide des fonctions **is_array**, **is_string**, **is_int**, **is_float**, **is_object**. (Les fonctions **is_double** et **is_real** sont équivalentes à **is_float**. Les fonctions **is_long** et **is_integer** sont équivalentes à **is_int**.)



Fonctions include() et require()



- La fonction **include()** inclut et exécute le fichier spécifié en argument
- Include est similaire à la fonction **require()**. Les deux structures de langage sont identiques, hormis dans leur gestion des erreurs. **include()** produit une Alerte (warning) tandis que **require()** génère une erreur fatale. En d'autres termes, n'hésitez pas à utiliser **require()** si vous voulez qu'un fichier d'inclusion manquant interrompe votre script. **include()** ne se comporte pas de cette façon, et le script continuera son exécution.
- Lorsqu'un fichier est inclus, le code qu'il contient hérite des variables qui existent dans ce contexte. Toute variable accessible à la ligne qui réalise l'exécution est aussi accessible dans le fichier inclus.
- Ne pas oublier qu'un fichier inclus peut lui-même contenir d'autres fichiers inclus...
- Éviter les extensions .inc qui présentent des risques de piratage : les fichiers ne sont généralement pas interprétés par le moteur PHP et risquent d'être envoyés **tels-quels** (s'ils sont appelés directement)



include_once(), require_once()



- La fonction **include_once()** inclut et évalue le fichier spécifié durant l'exécution du script. Le comportement est similaire à **include()**, mais la différence est que si le code a déjà été inclus, il ne le sera pas une seconde fois.
- La fonction **include_once()** est utilisée de préférence lorsque le fichier doit être inclus ou évalué plusieurs fois dans un script, ou bien lorsque vous voulez être sûr qu'il ne sera inclus qu'une seule fois, pour éviter des redéfinitions de fonction.

vars.php

```
<?php
$couleur = 'verte';
$fruit = 'pomme';
?>
```

test.php

```
<?php
echo "Une $couleur $fruit"; // Une
include ("vars.php");
echo "Une $couleur $fruit"; // Une verte pomme
?>
```

test.php

```
<?php
echo "Une $couleur $fruit"; // Une
include_once ("vars.php");
echo "Une $couleur $fruit"; // Une verte pomme
?>
```



Fonction die()



- La fonction die() permet de gérer les erreurs en PHP
- Elle permet de quitter le script en affichant le message d'erreurs
- Très pratique dans la gestion des erreurs de connexions à une base de données

```
$conec = mysql_connect(...) or die("Pb connexion...");
```